

jxl-java

安全なJPEG XLデコードを
すべてのJavaアプリへ

Rust × WebAssembly × Chicory AOT

ネイティブ依存なし・JNIなし・ImageIO互換

100% Pure Java

JARひとつ



JavaでJPEG XLを扱う“最後の壁”を取り除く

標準APIの使いやすさと、メモリ安全性・配布容易性を同時に実現

01 状況変化

**JPEG XLが
実用フェーズへ**

高画質・HDR・可逆再圧縮を1形式でカバー

02 課題

**Java標準は未対応
JNIは運用負荷が重い**

OS別バイナリ・クラッシュ面・
ネイティブ依存が残る

03 解決策

**RustをWasm化し
Javaバイトコードへ**

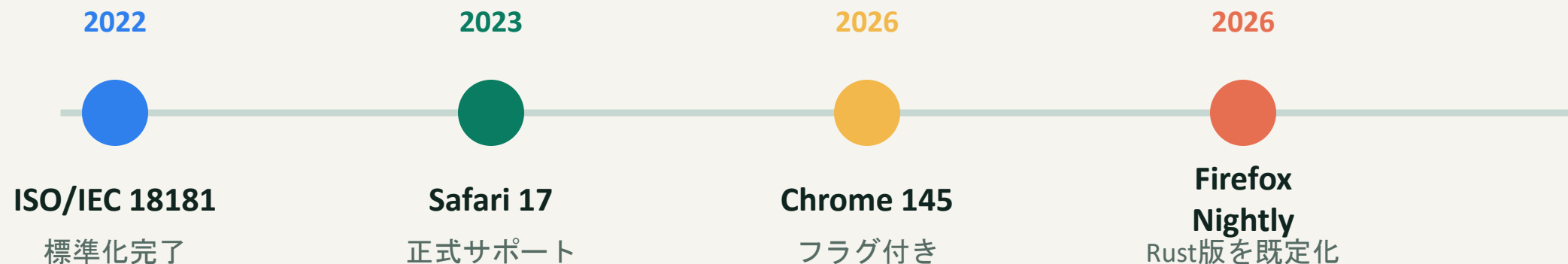
ImageIO.read()のまま
安全にデコード

本資料の結論

ピーク速度より安全性と可搬性を重視する用途で、jxl-javaは有力な選択肢

JPEG XL対応は進展中、普及はこれから

まだ“全ブラウザ対応済み”ではなく採用判断に備えるタイミング



JPEG XLの価値

高圧縮・高画質

可逆／非可逆

HDR・広色域

アルファ・アニメ

JPEG可逆再圧縮

プログレッシブ

JNI経由では画像処理のリスクがプロセス全体に...

従来方式は運用負荷と障害範囲がサーバー用途の足かせになる

従来 : libjxl + JNI



OS/CPU別バイナリ

クラッシュ時はプロセス全体に波及

jxl-java : Wasmサンドボックス

JVM

Rust decoder

異常はIOExceptionとして隔離

JNIなし

配布はJAR

全OS/CPU

外部から入力を扱うなら、速度だけでなく“障害範囲”と“配布運用”を含めての比較が必要

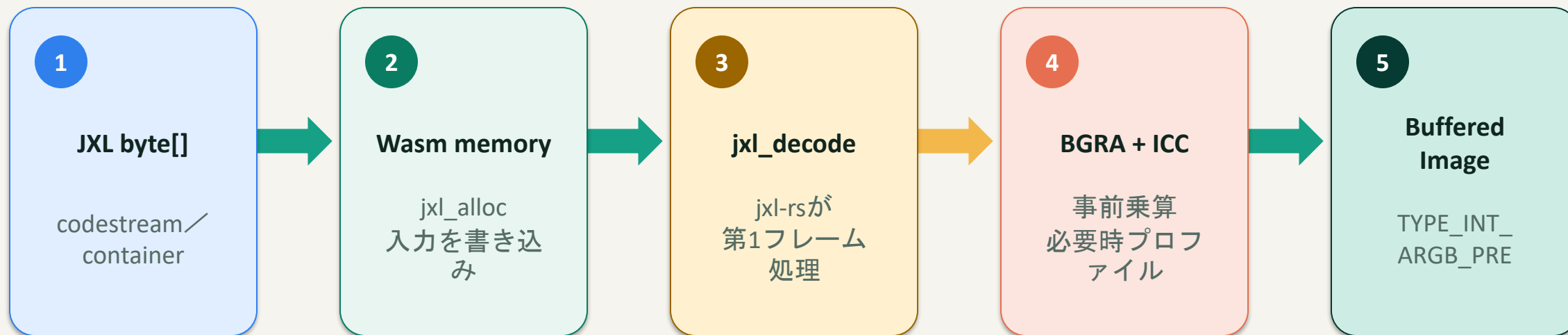
Rust製デコーダーを、ビルド時にJavaへ持ち込む

jxl-rs → WebAssembly → Chicory AOTの3段階



入力からBufferedImageまで、5段階で完全分離

変換回数を抑えつつ、デコード単位のインスタンスでスレッド安全性を確保



設計上のポイント

モジュール解析はJVMごとに1回。デコードごとに新規Wasmインスタンスを生成し、状態を共有しない。

導入後のコードは、いつものImageIOのまま

Service Provider Interfaceが自動登録され、初期化処理は不要

JAVA

```
BufferedImage image = ImageIO.read(new File("image.jpg"));
```

配布方式は2つ

STANDARD

通常版JAR

Chicory runtimeを依存に追加

SELF-CONTAINED

fat JAR

Chicoryを内部ヘシェーディング。衝突を回避

Runtime: Java 11+

Build: JDK 21+ & Rust

BGRAをそのままARGB intへ変換コストを最小化

リトルエンディアンのメモリ配置を利用し、ピクセル処理をコピー中心にする

Wasm出力 : 4 bytes



Java int

0xAARRGGBB

TYPE_INT_ARGB_PRE

事前乗算アルファ

Java 2Dで高速に合成

float精度で乗算

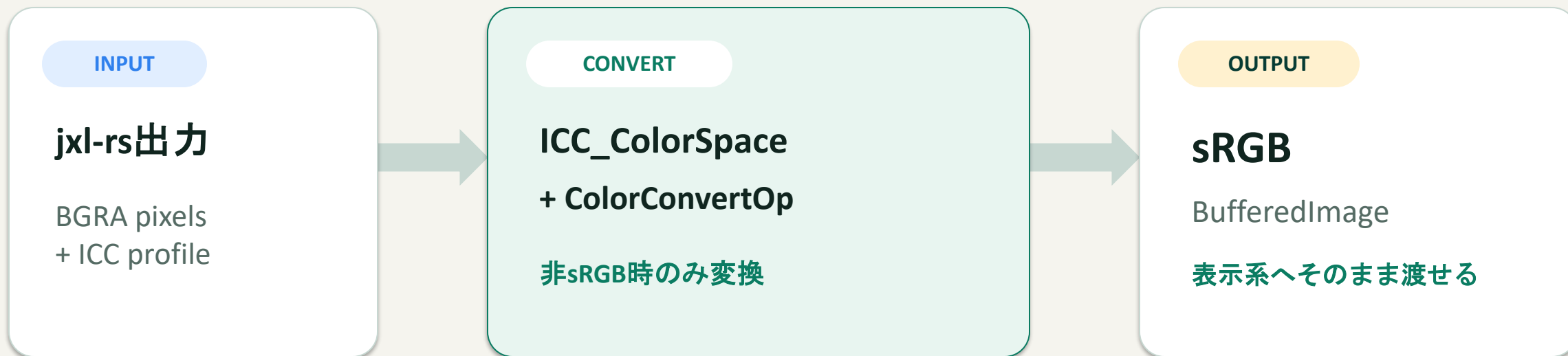
8bit変換パスを削減

メモリコピー中心

色順序変換を回避

広色域・ICC画像も、Java側でsRGBへ着地させる

“デコードできる”だけでなく、表示結果の一貫性まで扱う



壊れたプロファイルの場合

変換をスキップしてデコードを継続 & 画像全体の失敗を回避

速度は実用的。ただし大量バッチには向かない

現在はスカラーWasm実行。用途に合わせた期待値設定が重要

ウォームアップ後

1-2.5

秒 / メガピクセル

一般的なデスクトップ環境・元資料計測

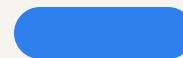
デコード以外

モジュール準備

インスタンス生成

ピクセル転送

ARGB組み立て



合計：数ms

今後の伸びしろ

Chicory AOTのWasm SIMD対応後は、jxl-rsのsimd128パスを活用して高速化できる可能性

安全性・可搬性を優先する読み込み用途にフィット

向いている

- ✓ サムネイル生成
- ✓ アップロード画像受け入れ
- ✓ デスクトップ表示
- ✓ マルチOS配布

向いていない

- × 大量バッチ変換
- × 低遅延が最優先
- × JPEG XLエンコード
- × 16bit／HDR保持

現在の制約

- 第1フレームのみ
- 8bit ARGB_PRE固定
- SIMD未使用
- 読み込み専用

推奨：まず非同期サムネイル生成など、速度要件を制御しやすい経路から導入